IPOS Integration Changes

CA ↔ SA ↔ PU Communication

Prepared by: IPOS-CA Team (Team 2)
Date: April 2026

Overview

This document lists every file changed to enable live HTTP communication between the three subsystems. It is split into three sections — one per team. Each entry states the file, what changed, and why.

The integration follows this architecture:

- **CA** → **SA**: CA places wholesale orders by POSTing to SA's REST API
- **SA** → **CA**: SA notifies CA of order status changes by POSTing to CA's embedded HTTP server
- **PU** → **CA**: PU notifies CA when an online sale completes by POSTing to CA's embedded HTTP server

CA runs natively on the local machine. SA runs on port 8080. PU runs on port 8082. CA's inbound server listens on port 8081.

IPOS-SA (Team 3) — 5 files changed

1. `backend/src/main/java/com/ipos/security/SecurityConfig.java`

What changed:

- `"/api/orders"` and `"/api/orders/**"` added to the CSRF ignore list
- `"http://localhost:8081"` added to the CORS allowed origins

Why:

CA is a Java desktop app — it does not run in a browser and has no CSRF cookie. Without exempting the orders endpoints from CSRF validation, every CA request to place or update an order would be rejected with 403. The CORS change ensures CA's HTTP client is not blocked by the browser-level origin policy.

```
// Before
.ignoringRequestMatchers("/api/auth/login")

// After
.ignoringRequestMatchers("/api/auth/login", "/api/orders", "/api/orders/**")
```

```
// Before
```

```
config.setAllowedOrigins(List.of("http://localhost:5173"));

// After
config.setAllowedOrigins(List.of("http://localhost:5173",
"http://localhost:8081"));
```

---

2. `backend/src/main/java/com/ipos/config/DataBootstrap.java`

What changed:

- A fourth entry added to the `bootstrapUsers` array: `ca_merchant` / `ca_pass` with `ROLE_MERCHANT`

Why:

CA authenticates with SA using session-based login. It needs a dedicated account on SA's system. This account is seeded automatically on first run (when `ipos.bootstrap.enabled=true`). CA calls `POST /api/auth/login` with these credentials on startup and stores the session cookie for all subsequent requests. SA's ORD-US1 enforcement means the merchantId in CA's order requests is automatically overridden to this account's ID.

```
// Added entry:
{"IPOS-CA System", "ca_merchant", "ca_pass", User.Role.MERCHANT},
```

---

3. `backend/src/main/java/com/ipos/entity/Order.java`

What changed:

- `DISPATCHED` and `DELIVERED` added to the inner `OrderStatus` enum

Why:

CA's wholesale order lifecycle requires these statuses to track an order through to delivery (at which point CA automatically increases its local stock). Without these values, SA cannot push meaningful status updates to CA and the delivery-triggers-stock-increase flow cannot work.

```
// Before
public enum OrderStatus { PENDING, CONFIRMED, CANCELLED }

// After
public enum OrderStatus { PENDING, CONFIRMED, DISPATCHED, DELIVERED, CANCELLED }
```

---

4. `backend/src/main/java/com/ipos/service/OrderService.java`

What changed:

- Added `updateOrderStatus(Long orderId, Order.OrderStatus newStatus)` method
- Added private `notifyCaStatusChange(Long orderId, Order.OrderStatus status)` method
- Added imports: `RestTemplate`, `Map`

Why:

SA needs a way to progress an order through its lifecycle (confirm it, mark it dispatched, mark it delivered). This method handles that and then pushes the new status to CA's inbound server at `http://localhost:8081/order-update`. The notification is wrapped in try-catch — if CA is offline, SA's own transaction still commits successfully.

```
@Transactional
public Order updateOrderStatus(Long orderId, Order.OrderStatus newStatus) {
    Order order = orderRepository.findById(orderId)
            .orElseThrow(() -> new RuntimeException("Order not found: " +
orderId));
    order.setStatus(newStatus);
    Order saved = orderRepository.save(order);
    notifyCaStatusChange(orderId, newStatus);
    return saved;
}

private void notifyCaStatusChange(Long orderId, Order.OrderStatus status) {
    try {
        RestTemplate rt = new RestTemplate();
        Map<String, Object> payload = Map.of("orderId", orderId, "status",
status.name());
        rt.postForObject("http://localhost:8081/order-update", payload,
String.class);
    } catch (Exception e) {
        System.err.println("[SA→CA] CA notification failed (non-fatal): " +
e.getMessage());
    }
}
```

---

5. `backend/src/main/java/com/ipos/controller/OrderController.java`

What changed:

- `PUT /api/orders/{id}/status` endpoint added
- `StatusUpdateRequest` record added
- Added imports: `PutMapping`, `PathVariable`

Why:

The SA admin UI (and SA team members testing) need a REST endpoint to update order status. This endpoint calls `orderService.updateOrderStatus()` which handles the status change and triggers the CA webhook automatically.

```
@PutMapping("/{id}/status")
```

```
public ResponseEntity<?> updateStatus(@PathVariable Long id,
                                      @RequestBody StatusUpdateRequest request) {
    try {
        Order updated = orderService.updateOrderStatus(id, request.status());
        return ResponseEntity.ok(updated);
    } catch (RuntimeException e) {
        return ResponseEntity.badRequest().body(Map.of("error", e.getMessage()));
    }
}

record StatusUpdateRequest(Order.OrderStatus status) {}
```

IPOS-PU (Team 1) — 2 files changed

1. `src/main/resources/application.properties`

What changed:

- `server.port=8082` added as the first line

Why:

PU was defaulting to port 8080 which clashes with SA. All three systems need to run simultaneously on the same machine during the demo. CA's inbound server is on 8081, SA stays on 8080, PU moves to 8082.

```
server.port=8082
```

---

2. `src/main/java/com/ipos/pu/service/OrderService.java`

What changed:

- Lines 72–73 (the `// Notify IPOS-CA (mock)` comment and `System.out.println`) replaced with a real `RestTemplate` POST to `http://localhost:8081/online-sale`
- Added imports: `RestTemplate`, `LocalDate`, `ArrayList`, `LinkedHashMap`, `Map`

Why:

This was the placeholder that was always intended to become a real call ("added in Week 5" per the original comment). The payload matches the format CA's inbound server expects: `puOrderId`, `receivedDate`, `customerEmail`, and an `items` array of `{itemId, quantity}` pairs. The item IDs sent are PU's `Product.id` values — these must match CA's stock item IDs 1–5 (see agreement note below). Failure is non-fatal so a CA outage never breaks PU's order flow.

```
// Before (lines 72-73):
// Notify IPOS-CA (mock for now — real RestTemplate call added in Week 5)
System.out.println("IPOS-CA: Deducting stock for order " + savedOrder.getId());
```

```
// After:
try {
    RestTemplate rt = new RestTemplate();
    Map<String, Object> payload = new LinkedHashMap<>();
    payload.put("puOrderId", "PU-" + savedOrder.getId());
    payload.put("receivedDate", LocalDate.now().toString());
    payload.put("customerEmail", member.getEmail());
    List<Map<String, Object>> saleItems = new ArrayList<>();
    for (CartItem ci : cartItems) {
        Map<String, Object> saleItem = new LinkedHashMap<>();
        saleItem.put("itemId", ci.getProduct().getId());
        saleItem.put("quantity", ci.getQuantity());
        saleItems.add(saleItem);
    }
    payload.put("items", saleItems);
    rt.postForObject("http://localhost:8081/online-sale", payload, String.class);
} catch (Exception e) {
    System.err.println("[PU→CA] CA notification failed (non-fatal): " +
e.getMessage());
}
```

IPOS-CA (Team 2) — 9 files changed, 4 files created

These changes are already committed to the CA repository. Listed here for completeness.

New Files

File	Purpose
`integration/HttpSaGateway.java`	Real HTTP implementation of `ISaGateway`. Logs in to SA on startup, POSTs orders to `/api/orders`, stores SA's returned orderId locally. Swaps in for `MockSaGateway` when running with `-Dipos.http=true`.
`integration/CaApiServer.java`	Embedded JDK `HttpServer` on port 8081. Handles `POST /order-update` (from SA) and `POST /online-sale` (from PU). Calls service layer then fires UI refresh events on the EDT.
`app/AppContext.java`	Static service locator and event bus. Holds shared service instances. UI screens register refresh listeners here instead of constructing their own services.
`integration/LocalDateAdapter.java`	Gson type adapter for `LocalDate` ↔ ISO-8601 strings. Required because Gson does not handle `java.time` types natively.

Modified Files

File	What changed
`model/WholesaleOrder.java`	Added `saOrderId` field (non-final, with getter/setter) to store SA's assigned order ID after submission

`model/OrderStatus.java`	Added `CANCELLED` to the enum to match SA's status vocabulary
`db/DatabaseManager.java`	Migration: `ALTER TABLE wholesale_orders ADD COLUMN sa_order_id` added (safe to run repeatedly — skipped if column exists)
`repository/WholesaleOrderRepository.java`	Added `findBySaOrderId()` and `updateSaOrderId()` to the interface
`repository/WholesaleOrderRepositoryImpl.java`	Implemented both new methods; `save()` and `mapRow()` updated to include `sa_order_id` column
`service/WholesaleOrderService.java`	Added `receiveStatusUpdate(int saOrderId, OrderStatus status)` — called by `CaApiServer` when SA pushes a status change
`app/Main.java`	Reads `-Dipos.http=true` flag; wires shared services; initialises `AppContext`; starts `CaApiServer` in HTTP mode; registers shutdown hook
`ui/WholesaleOrderUI.java`	Uses `AppContext` for services instead of constructing them locally; `handlePlaceOrder` wrapped in `SwingWorker` so HTTP calls don't block the UI; registers order refresh listener
`ui/StockManagementUI.java`	Registers stock refresh listener so table auto-updates when PU pushes an online sale

---

Running the Integrated System

Prerequisites — one-time setup

1. Create the SA database:

```
/usr/local/mysql/bin/mysql -u root -p -e "CREATE DATABASE IF NOT EXISTS ipos_sa;"
```

2. Seed SA's product catalogue (SA has no products by default — CA orders will fail without this):

```
/usr/local/mysql/bin/mysql -u root -p ipos_sa <<'EOF'
INSERT INTO products (id, product_code, description, price, availability_count,
min_stock_threshold) VALUES
(1, 'PARA-500', 'Paracetamol 500mg', 2.50, 500, 50),
(2, 'IBUP-200', 'Ibuprofen 200mg',   3.20, 400, 40),
(3, 'AMOX-250', 'Amoxicillin 250mg', 8.75, 200, 20),
(4, 'CETI-10',  'Cetirizine 10mg',   4.10, 300, 30),
(5, 'OMEP-20',  'Omeprazole 20mg',   5.60, 250, 25)
ON DUPLICATE KEY UPDATE product_code=VALUES(product_code);
EOF
```

These IDs match CA's stock item IDs 1–5 exactly. This only needs to be run once — the rows persist across restarts.

Startup order

1. Start MySQL
2. Start SA: `mvn spring-boot:run` (port 8080)
3. Start PU: `mvn spring-boot:run` (port 8082)
4. Compile CA (first time only): `find src -name "*.java" > sources.txt && javac -cp "lib/*" -d out @sources.txt`
5. Start CA with HTTP flag: `java -Dipos.http=true -cp "out:lib/*" app.Main`

CA logs `[HttpSaGateway] Logged in to SA successfully` and `[CaApiServer] Listening on port 8081` on successful startup.

Mock mode (no SA/PU needed)

```
java -cp "out:lib/*" app.Main
```

All SA and PU behaviour is simulated locally. The demo works fully offline.

Verify the integration is working

```
# Test 1 — PU → CA: simulate an online sale (deducts 5 Paracetamol from CA stock)
curl -X POST http://localhost:8081/online-sale \
  -H "Content-Type: application/json" \
  -d '{"puOrderId":"PU-TEST","receivedDate":"2026-04-
12","customerEmail":"test@test.com","items":[{"itemId":1,"quantity":5}]}'
# Expected: {"accepted":true,"fullyApplied":true}

# Test 2 — SA → CA: simulate a status update push
curl -X POST http://localhost:8081/order-update \
  -H "Content-Type: application/json" \
  -d '{"orderId":1,"status":"CONFIRMED"}'
# Expected: {"ok":true}

# Test 3 — CA → SA: place a wholesale order from within the CA app
# Use the Wholesale Orders screen in CA — if SA products are seeded and CA
# shows "SA confirmed — SA Order ID: X", the link is working end to end.
# Verify the order landed in SA's database:
/usr/local/mysql/bin/mysql -u root -p -e "USE ipos_sa; SELECT * FROM orders;"
```

Important: Product ID Agreement

Product IDs are now confirmed and locked. SA's products were seeded manually (see Prerequisites above) to match CA's stock IDs exactly.

CA itemId	Item Name	SA productId	SA productCode	PU productId
1	Paracetamol 500mg	1	PARA-500	TBC
2	Ibuprofen 200mg	2	IBUP-200	TBC
3	Amoxicillin 250mg	3	AMOX-250	TBC
4	Cetirizine 10mg	4	CETI-10	TBC
5	Omeprazole 20mg	5	OMEP-20	TBC

CA ↔ SA product ID mapping is confirmed. PU product IDs still need to be verified with Team 1.